



ELECTRONICS AND CONTROLS

WEEK 1 - SUBMISSION

By:- Arka Dasgupta (2025ACPS1266G)

[21 June 2026]

~ ASSIGNMENT 1 ~

Assignment Name:- LED Control using Serial - RGB Potentiometer

Link to Assignment (Tinkercad):-

<https://www.tinkercad.com/things/hZEVfKI948e-led-control-using-serial-rgb-potentiometer?sharecode=qAf3SEcgavgo6rMw0LwHhZvZkhWV4olbMIL1pVdwIYU>

Hardware Used:-

1. **MCU Used** → Two Arduino Uno R3
2. **Key Components:-**
 - 1x Potentiometer
 - 1x RGB LED
 - 3x Current-limiting Resistors (1kΩ)

Implementation Strategy:-

1. **Read and Scale:** The system continuously reads the analog voltage from the potentiometer (0–4095 on an ESP32 ADC) and uses a mapping function to scale it down to an 8-bit range (0–255) suitable for PWM control.
2. **Smooth Output:** The scaled value is then directly written to the LED pin using PWM (Pulse Width Modulation) to dynamically adjust its brightness based on the physical dial's position.

Custom Function Description:-

None. This project relies entirely on standard Arduino `setup()` and `loop()` structures. No custom user-defined functions were created.

Known Bugs & Edge Cases:-

None.

~ ASSIGNMENT 2 ~

Assignment Name:- Obstacle Detector

Link to Assignment (Tinkercad):-

https://www.tinkercad.com/things/j7PQqYKvuul-obstacle-detector-light-buzzer?s harecode=kg3h4iCbLYvxuAyA-UXSr8_S6m7WT3cvDL-F2Eqmx40

Hardware Used:-

1. **MCU Used** → Two Arduino Uno R3
2. **Key Components:-**
 - 1x HC-SR04 Ultrasonic Distance Sensor,
 - 1x 16x2 I2C LCD Display
 - 1x LED
 - 1x Piezo Buzzer
 - 1x Current-limiting Resistor (1kΩ)

Implementation Strategy:-

1. **Distance Tracking:** The system triggers the HC-SR04 ultrasonic sensor using a precise 10-microsecond pulse, tracks the wave's return time via `pulseIn()`, and translates this duration into centimeters using the standard speed of sound formula:

$$\text{distance} = (\text{duration} * 0.034 / 2)$$
2. **Threshold-Driven Alerts:** The current distance is continuously output to the top row of the LCD screen. If an object is detected closer than 50cm, the program shifts into an alarm state that prints a warning message on the bottom row, scrolls the text, blinks the LED, and sounds a 500 Hz tone on the piezo buzzer.

Custom Function Description:-

None. This project relies entirely on standard Arduino `setup()` and `loop()` structures. No custom user-defined functions were created.

Known Bugs & Edge Cases:-

None.

~ ASSIGNMENT 3 ~

Assignment Name:- Emergency Push Button (E-Stop)

Link to Assignment (Wokwi):-

<https://wokwi.com/projects/467106869021225985>

Hardware Used:-

1. **MCU Used** → ESP32-S3
2. **Key Components:-**
 - 1x Push Button
 - 1x LED (Red)

Implementation Strategy:-

1. **Continuous Polling:** The program uses the internal pull-up resistor configuration (`INPUT_PULLUP`) to keep the button pin high by default. The main `loop()` continuously polls this pin via `digitalRead()` to check if it drops to `LOW` (indicating the button has been pressed).
2. **Hardware Freeze Lockout:** Once a press is registered, the system triggers the safety state by lighting up the red indicator LED, printing a panic alert to the Serial monitor, and forcing the ESP32 into an infinite `while(true)` loop that completely locks down the processor to stop all rover operations.

Custom Function Description:-

None.

Note: This project relies entirely on standard Arduino `setup()` and `loop()` structures. No custom user-defined functions were created.

Known Bugs & Edge Cases:-

None.

~ ASSIGNMENT 4 ~

Assignment Name:- Four Wheel Independent Steer

Link to Assignment (Wokwi):-

<https://wokwi.com/projects/467440104105540609>

Hardware Used:-

1. MCU Used → ESP32 (standard)
2. Key Components:-
 - 1x Analog Joystick
 - 4x Servo Motors

Implementation Strategy:-

1. Read the 12-bit analog input (0 to 4095) from the joystick's X-axis and used the `map()` function to proportionally convert it into a 0 to 180-degree steering angle.
2. Implemented a software debounce for the joystick's select button to safely toggle a boolean state variable. Depending on this state, the program either sends the exact same angle to all four servos (Mode 1: Crab Walk) or subtracts the front angle from 180 to generate an inverted angle for the rear servos (Mode 2: Point Turn).

Custom Function Description:-

None.

Note: This project relies entirely on standard Arduino `setup()` and `loop()` structures. No custom user-defined functions were created.

Known Bugs & Edge Cases:-

None.

~ ASSIGNMENT 5 ~

Assignment Name:- Encoder Interfacing

Link to Assignment (Wokwi):-

<https://wokwi.com/projects/467450139225497601>

Hardware Used:-

1. **MCU Used** → ESP32-S3
2. **Key Components:-**
 - 1x Analog Joystick
 - 1x Rotary Encoder Module (with CLK, DT, VCC and GND pins)

Implementation Strategy:-

1. **Hardware-Driven Decoding:** Solved the task by leveraging the ESP32-S3's dedicated internal Pulse Count Controller (PCNT) peripheral via the `ESP32Encoder` library. This decodes the quadrature signal phases (CLK and DT) entirely in hardware background threads without relying on performance-heavy software interrupts.
2. **State-Change Monitoring:** The main execution loop reads the hardware register count and uses conditional tracking logic to instantly output the current angular position to the Serial Monitor only when a physical positional change is detected.

Custom Function Description:-

None.

Note: This project relies entirely on standard Arduino `setup()` and `loop()` structures. No custom user-defined functions were created.

Known Bugs & Edge Cases:-

None.

~ ASSIGNMENT 6 ~

Assignment Name:- PID Controller to Stabilize Inverted Pendulum

Link to Assignment (Google Colab):-

<https://colab.research.google.com/drive/1HgdIt6RAa21nnOYI8Od61kbGx2Reum3e?usp=sharing>

 ArkaDasgupta_Week1_ProjectKratos_PIDControlInvertedPendulum.ip...

Materials Used:-

1. **Software Simulation run on Google Colab Notebook using Python**
2. **Key Components:-** Simulated DC motor actuator (applying torque upto ± 20.0 Nm), simulated rotary encoder/sensor (measuring pendulum angle with Gaussian noise).

Implementation Strategy:-

1. Designed a discrete-time PID controller that dynamically calculates the required corrective motor torque to balance the pendulum at the upright vertical position (0.0 radians).
2. Used a high Proportional gain ($K_p = 35.0$) to aggressively counteract gravity, balanced by a Derivative gain ($K_d = 8.0$) to act as a damper to prevent wild overshoots and a small Integral gain ($K_i = 0.5$) to eliminate steady-state errors caused by random sensor noise.

Custom Function Description:-

```
def __init__(self) -> None:
    """
```

Initializes the PID controller by defining the tuning gains (K_p , K_i , K_d) and preparing state variables to track error over time.

Args:

None

Returns:

None

```
"""
```

Pass

```
def compute(self, setpoint: float, current_angle: float, dt: float) -> float:
    """
    Calculates the motor torque required to balance the pendulum using discrete PID logic.
    Args:
        setpoint: The target angle in radians (0.0 represents perfectly upright).
        current_angle: The current physical angle of the pendulum in radians (includes
noise).
        dt: The time elapsed since the last physics simulation frame, in seconds.
    Returns:
        float: The total calculated control signal (torque) to be applied to the motor.
    """
    pass
```

Known Bugs & Edge Cases:-

1. **Maximum Torque Saturation:** The physics simulation clips the motor output torque at ± 20.0 Nm. If the pendulum falls past a critical angle where gravity requires more than 20.0 Nm of torque to recover, the controller will saturate and the pendulum will fall completely over, triggering a system reset.
2. **Sensor Noise Vulnerability:** The simulation injects random noise into the `current_angle`. If the derivative gain (Kd) is tuned too high, it can amplify this high-frequency noise, causing rapid, erratic chatter in the motor output torque.